# Tugas Kelompok 10: Implementasi WAR, RAW, dan WAW pada Simulator ILP

Organisasi dan Arsitektur Komputer


**Anggota Kelompok 10:**
• Riswan / 51248190
• Fahrul Putra Irawan / 51248121
• Dzikra Rahika Farhan / 51248117


**Link GitHub Repository:**
https://github.com/riswan-51248190/tugas-ilp-kelompok10

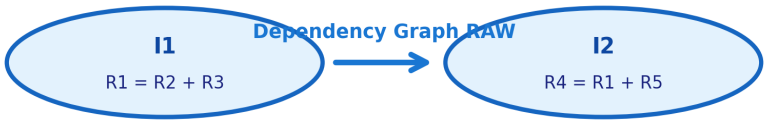
## Pendahuluan

Simulator ILP awal hanya mendeteksi satu jenis dependensi, yaitu RAW (inst2 membaca register yang ditulis inst1). Pada tugas ini fungsi *has_dependency* diperluas menjadi *dependency_types* yang mengenali tiga jenis data hazard. Dalam kode, I1 selalu mendahului I2 pada urutan program.

| Hazard | Kondisi | Contoh |
|---|---|---|
| RAW | I2 membaca register yang ditulis I1 | I1: R1 = R2 + R3<br>I2: R4 = R1 + R5 |
| WAR | I2 menulis register yang dibaca I1 | I1: R1 = R2 + R3<br>I2: R2 = R4 + R5 |
| WAW | I1 dan I2 menulis register yang sama | I1: R1 = R2 + R3<br>I2: R1 = R4 + R5 |

## 1. Skenario RAW

**Instruksi:**
I1: R1 = R2 + R3
I2: R4 = R1 + R5



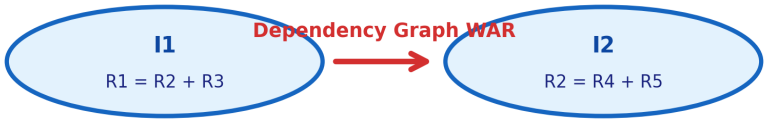

| Instruksi | Hazard | Cycle (tanpa renaming) | Cycle (dengan renaming) |
|---|---|---|---|
| I2 terhadap I1 | RAW | 2 | 2 (tidak berubah) |

**Analisis:** I2 membaca R1 yang baru ditulis I1, sehingga I2 harus menunggu I1 selesai. Ini dependensi data asli (*true dependence*) dan tidak bisa dihilangkan dengan register renaming. Hasil simulasi: 2 cycle, sama dengan eksekusi sekuensial.

## 2. Skenario WAR

**Instruksi:**
I1: R1 = R2 + R3

I2: R2 = R4 + R5

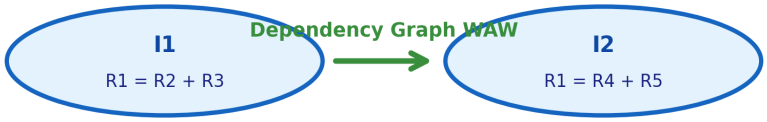



| Instruksi | Hazard | Cycle (tanpa renaming) | Cycle (dengan renaming) |
|---|---|---|---|
| I2 terhadap I1 | WAR | 2 | 1 |

**Analisis:** I2 menulis R2 yang masih dibaca I1 (*anti-dependence*). Tidak ada aliran data antara kedua instruksi, hanya nama register yang sama. Jika dijalankan dengan semua hazard dihormati, simulator memakai 2 cycle. Setelah register renaming, kedua instruksi dapat berjalan paralel dalam 1 cycle.

## 3. Skenario WAW

**Instruksi:**
I1: R1 = R2 + R3
I2: R1 = R4 + R5




| Instruksi | Hazard | Cycle (tanpa renaming) | Cycle (dengan renaming) |
|---|---|---|---|
| I2 terhadap I1 | WAW | 2 | 1 |

**Analisis:** I1 dan I2 sama-sama menulis R1 (*output dependence*). Jika I2 selesai lebih dulu, nilai akhir R1 menjadi salah. Tanpa renaming simulator memakai 2 cycle; dengan renaming cukup 1 cycle.

## Kesimpulan

RAW adalah dependensi yang sebenarnya karena ada aliran data, sedangkan WAR dan WAW hanya *name dependence* yang dapat dihilangkan dengan register renaming. Dengan renaming, jumlah cycle pada skenario WAR dan WAW turun dari 2 menjadi 1, sementara skenario RAW tetap 2 cycle. Kode lengkap dan riwayat commit setiap anggota tersedia pada repositori GitHub yang tercantum di halaman pertama.